ROBO 5000 Challenge Problem 2

Luke McSherry and Clint Lohr

University of Colorado Boulder, Boulder CO 80309, USA

1 Introduction

This document describes an implementation and display of a nonholomonic rapidly-exploring random tree (RRT) search in the Pybullet environment to assist with bringing a given "Turtlebot" to its goal. Here, we will describe the choices that led us to our final version of code, as well as its resulting videos.

2 Algorithm Development

2.1 Rapidly-Exploring Random Tree (RRT) Search

For this project we we assigned to implement a Rapidly Exploring Random Tree (RRT) algorithm to navigate a differential-drive Turtlebot through an arena towards a goal area. The arena included multiple obstacles of which the Turtlebot needed to find a path around. Instead of implementing the RRT algorithm from previous assignments, we attempted to simplify the algorithm by creating a RRTNode class. Each instance of this class represented a node in the path and was used to support the search by storing its position (x, y, z coordinates), quaternion orientation (x, y, z, w), parent pointer, the control action (velocity left, velocity right, duration), and the trajectory points which stores the movement trajectory over iterations.

Because the Turtlebot is subject to non-holonomic constraints and cannot navigate sideways instantaneously, the tree is expanded using dynamic control inputs rather than straight lines. For this simulation we set a 50% goal bias, meaning that at each iteration the algorithm samples the actual goal state 50% of the time and a random point in the arena the other times. Intermediate goal markers are updated with cross symbols and identify the nearest node in the tree (Figure 1). The planner then calculates the Euclidean distance between this sampled point and every node currently in the tree, selecting the closest existing node as the point for expansion.

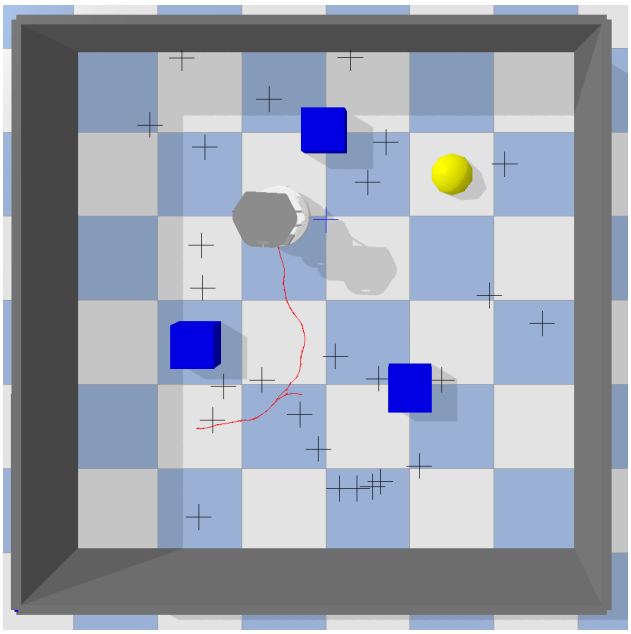


Fig. 1. Top-down view showing the Turtlebot navigating around static blue obstacle blocks toward the yellow goal sphere. Red curves represent the valid trajectories comprising the search tree branches. Black crosses mark previously sampled intermediate target points, while the blue cross highlights the currently active sample point.

From this nearest node, the simulation integrates five rollouts with random wheel velocities over a short duration. Any rollouts that collide with arena boundaries or obstacles are discarded. Valid candidate rollouts are plotted in red to visualize the explored branches, the rollout whose final endpoint gets closest to the sampled target is selected and it added as a new node in the tree, create the Monte Carlo Propagation.

Once a rollout is within the 0.3 meter radius of the goal, the Turtlebot is said to have reached its goal and the search concludes.

3 PyBullet Display

3.1 Drawing Functionality

For drawing the lines that display the robot's path as it moves, We drew a red path whenever a path was chosen to explore. Additionally, when drawing the crosses to show intermediate targets, we manage to set the previous cross to turn from blue to black, and then drawing a new blue cross to describe an intermediate target. We utilize the makeup of our `old_goals` data structure to do each intermediate target drawing operation in constant time.

Finally, upon reaching the target, a green line is drawn from the goal to the starting location of the Turtlebot. `search.mp4`'s last 5 seconds will display this green line.

3.2 Visualizing the Solution

For the visualization of the successful path, the winning trajectory is extracted by backtracking through parent node references from the goal to the root. The trajectory points are stitched together and displayed as a green path overlaying the red exploration tree (Figure 2).

To deploy and verify the solution, the simulation resets the Turtlebot back to its initial start position and resets all velocities. The robot then executes the exact sequence of wheel velocities and step durations in order, tracking the green path across the arena until it reaches the goal and stops. This solution path is captured using PyBullet's STATE_LOGGING_VIDEO_MP4 functionality which creates a video of Turtlebot navigating the successful path. This video is saved in the root directory of the source code as `final_path.mp4`

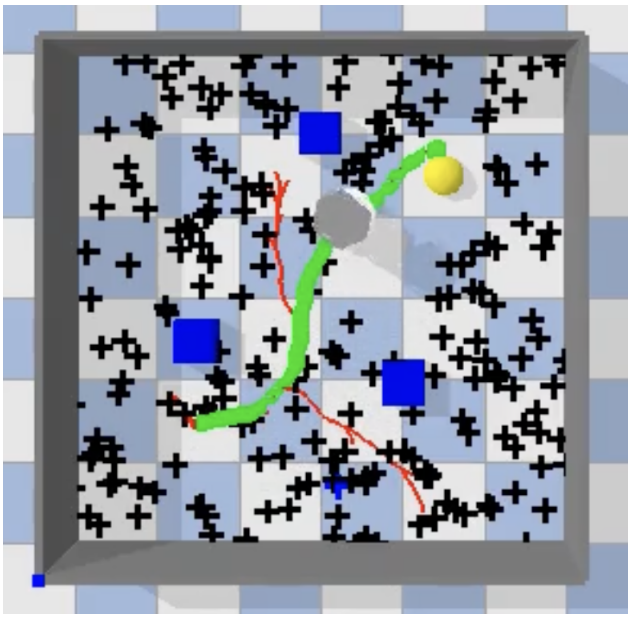


Fig. 2. Top-down view showing a screenshot from the final path video. The Turtlebot is following the green path representing the solution path from root to goal.

3.3 OpenGL issues on MacOS

Upon completing our algorithm and drawing functionality, our result was not drawing a clear green line. Additionally, all our drawings in general appeared hard to view. In particular, the lines appeared very thin, which made them difficult to interpret. This was due to us both using a Mac, which has documented incompatibilities with OpenGL drawing functions. To address this, we pulled the CP2 repo onto a Windows desktop computer and ran it there while iterating on our drawing functions, drastically improving the resulting videos.

References

1. Bullet Physics Community, "Real-Time Physics Simulation Forum — Topic 11976," *pybullet.org*. [Online]. Available: https://pybullet.org/Bullet/phpBB3/viewtopic.php?t=11976. [Accessed: Oct. 18, 2026].
2. andytgl, "What is the sequence of rotations applied by "getEulerFromQuaternion"? — Real-Time Physics Simulation Forum," *pybullet.org*. [Online]. Available: https://pybullet.org/Bullet/phpBB3/viewtopic.php?t=13235. [Accessed: Oct. 11, 2026].